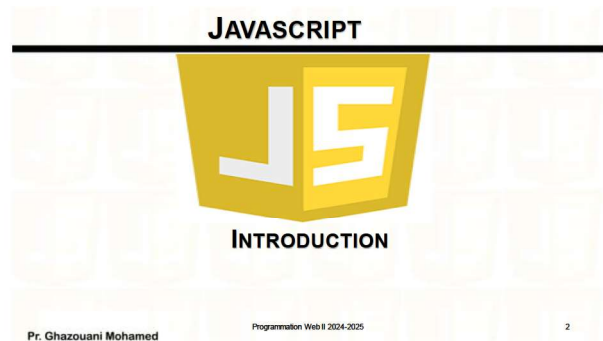




1



2

Javascript: c'est quoi ?

- Langage de script pour les pages web interactives
- Côté client : les scripts sont exécutés par le navigateur de l'utilisateur
- HTML pour le contenu, CSS pour la présentation, JavaScript pour programmer...
 - côté client seulement !
 - c'est-à-dire sur la page web récupérée par l'utilisateur (mais toujours même page récupérée)
- **Que peut-on faire en Javascript ?**
 - Puissance d'un langage de programmation complet, notamment structures conditionnelles (if/then/else), et boucles pour les répétitions (for, while).
 - Possibilité de changer le contenu de la page (modifier le HTML), les styles (modifier le CSS), d'afficher des boîtes de dialogue (messages d'alerte), de récupérer les événements clavier et souris (et réagir en conséquence).

3

Javascript: c'est quoi ?

Quelques exemples courants d'utilisation de JavaScript :

- Afficher des boîtes de dialogue pour confirmer une action
- Vérifier en temps réel (sans soumettre) que les informations sont valides lorsqu'on remplit les champs d'un formulaire (le mot de passe a le bon nombre de caractères, téléphone = que des chiffres...)
- Suggestions en temps réel dans votre moteur de recherche comme le fait Google !
- Un menu déroulant dynamique, un graphique qui montre l'évolution de quelque chose en temps réel (cours de la bourse par exemple), etc.

4

Javascript: à quoi ça ressemble ?

```
<!DOCTYPE html>
<html>
<script>
function maFonction()
{
let personne;
personne = prompt("Entrez votre nom", "Toto");
if (personne !== null && personne.trim() !== "")
{
document.getElementById("demo").innerHTML = "Bonjour, "
+ personne + " !";
}
}
</script>
<body>
<p>Cliquez sur le bouton pour appeler la fonction !</p>
<button onclick="maFonction()">Appel</button>
<p id="demo"></p>
</body>
</html>
```



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025



5

Insérer du javascript dans une page web

- Les script sont dans les balises `<script>...</script>`
- Avant HTML 5, il fallait utiliser `<script type="text/javascript">...</script>`. Le paramètre type n'est plus nécessaire maintenant, vous pouvez l'omettre.
- Possible de mettre des balises script multiples, dans le header ou le body (sans importance)
- Pour éditer vos scripts : n'importe quel éditeur de texte, comme pour du HTML ou du CSS.
- Au lieu d'ajouter le code directement dans la page, possible de le mettre dans un fichier séparé (généralement avec l'extension .js) et de l'inclure avec: `<script src="chemin/vers/fichier.js"></script>`

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

6

Insérer du javascript dans une page web

- Deux possibilités pour un élément `<script>`
- soit il contient des instructions JavaScript ①
 - soit il pointe vers un fichier de script externe au travers de son attribut `src` ②
 - url fichier externe, permet de localiser un fichier texte qui contient le code JavaScript.
 - url vers un fichier du site : on indique le `./chemin`
 - url vers un fichier d'un site externe: `http:// ....`
 - fonctionnement équivalent à une intégration directe du code JavaScript à cet endroit.

Quand l'attribut `src` est utilisé il ne faut rien écrire entre les balises `<script src="...">` et `</script>`



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

7

7

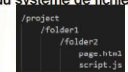
Insérer du javascript dans une page web

1. Chemin Absolu

Définition: C'est le chemin complet à partir de la racine du système de fichiers jusqu'au fichier ou au dossier.

Exemple:

C:\Users\Utilisateur\Documents\monscript.js



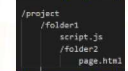
`<script src="script.js"></script>`

2. Chemin Relatif

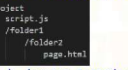
Définition: C'est le chemin par rapport à la position du fichier courant (le fichier dans lequel le chemin est utilisé).

Exemple:

../monscript.js signifie : aller dans le répertoire parent, puis chercher "monscript.js"



`<script src="../script.js"></script>`



`<script src="../../script.js"></script>` 8

Pr. Ghazouani Mohamed

8

JavaScript : éléments de base

- Toutes les instructions doivent être séparées par des points-virgule (semicolon en Anglais) :

```
console.log("Hello, world !");
console.log("J'écris dans la console JavaScript");
alert("Voici une alerte !");
document.write("bonjour tout le monde");
```

- console.log() permet d'écrire dans la console JavaScript, pratique pour déboguer vos programmes (afficher la valeur d'une variable, ou voir si on est entré dans un if, par exemple)

- Pour afficher la console JavaScript :

Firefox : Menu Outils > Outils de navigateur > Développement de développement web > Console web

Sous Chrome, Edge et Firefox : F12

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

9

JavaScript dans un navigateur

- Code JavaScript peut être inclus dans des pages HTML

En quoi JavaScript est-il si différent d'autres langages comme PHP ?

HelloWorld.html

```
<!DOCTYPE html>
<html>
<head>
<title>Hello world/</title>
</head>
<body>
<h1>Hello/</h1>
<script>
for (i=0; i < 5; i++) {
document.write("Hello World !<br>");
}
</script>
</body>
</html>
```

HelloWorld.php

```
<!DOCTYPE html>
<html>
<head>
<title>Hello world/</title>
</head>
<body>
<h1>Hello/</h1>
<?php
for ($i=0; $i < 5; $i++) {
echo "Hello World !<br>";
}
}
</body>
</html>
```

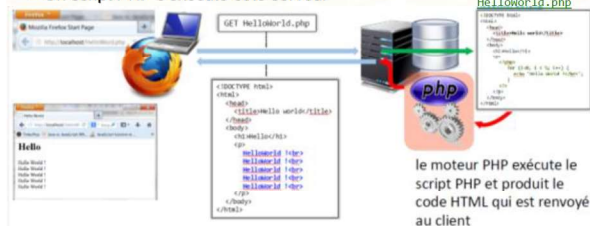
Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

10

JavaScript dans un navigateur

- Un script PHP s'exécute côté serveur



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

11

11

JavaScript dans un navigateur

- Un script JavaScript s'exécute côté client (dans le navigateur)



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

12

12

JAVASCRIPT



ÉLÉMENTS DE BASE

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

13

13

JavaScript : éléments de base

- Commentaires dans le code : soit entre `/*` et `*/` (lignes multiples possibles)
- Soit après `//` jusqu'à un retour à la ligne
- Rappel : tout ce qui est entre commentaire ignoré lors de l'exécution du code. Utile pour documenter le code, ou pour en « enlever » temporairement une partie quand vous travaillez dessus
- Même chose qu'en PHP, C++ ou Java par exemple...
- Exemple :


```
/* Je peux revenir
à la ligne facilement
comme ça pour les longs commentaires */
console.log("Hello World!"); // Commentaire court
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

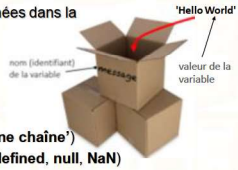
14

14

JavaScript : éléments de base

Variables

- Utilisées pour stocker temporairement des données dans la mémoire de l'ordinateur
- En JavaScript deux types de valeurs :
 - valeurs primitives
 - notation littérales
 - ◊ de nombres (14, 3.14159, 3.25e-6)
 - ◊ de chaînes de caractères ("Ceci est une chaîne")
 - valeurs natives (built-in) (true, false, undefined, null, NaN)
 - objets
 - c'est-à-dire des conteneurs (par exemple un tableau), la valeur de la variable dans ce cas est l'adresse (référence) du



JavaScript : éléments de base

Variables : règles de nommage

- une variable doit être désignée avec un nom unique (dans sa portée) : identifiant
- règles similaires à celles de la plupart des langages :
 - noms ne peuvent contenir d'espace, d'opérateurs arithmétiques (+, -, /, *) ou de caractère de ponctuation (.,:)
 - noms peuvent contenir des lettres, chiffres, _ ou \$
 - noms ne peuvent commencer par un chiffre
 - noms sensibles à la casse (case sensitive) : maVariable vs mavariable
 - noms doivent être différents des mots réservés (par ex. var, if, while, function...).

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

16

16

JavaScript : éléments de base

Variables : règles de nommage (Suite)

Bonnes pratiques

- choisir des noms descriptifs
 - par exemple pour une variable qui correspond à une moyenne on préférera **moyenne** à **m**
- ne pas hésiter à utiliser plusieurs mots pour décrire avec précision le rôle d'une variable
 - par exemple pour une variable qui correspond à un prix unitaire d'un produit on préférera **prixUnitaire** à **pu**
- adopter des règles de nommage cohérentes dans tout le code

Définir un glossaire, réutiliser les mêmes termes partout Conserver les mêmes conventions d'écriture



17

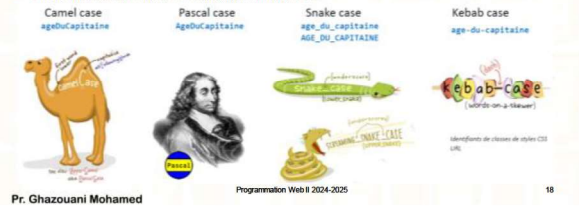
JavaScript : éléments de base

Variables : règles de nommage (Suite)

- Comment regrouper plusieurs mots dans un identifiant

Ex : variable représentant l'âge du capitaine.

- Conventions d'écriture couramment utilisées



18

JavaScript : éléments de base

Déclaration des variables

- historiquement déclaration par **var** *nomDeVariable*;
- mais depuis ES6 (ECMAScript 2015) la bonne pratique est d'utiliser **let** au lieu de **var**



- on peut même utiliser une variable sans la déclarer !

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

19

19

JavaScript : éléments de base

Portée (Scope) :

var :

- A une portée globale si déclarée hors d'une fonction.
- A une portée fonction si déclarée à l'intérieur d'une fonction, peu importe où elle se trouve dans la fonction.
- N'est pas limitée par les blocs ({ ... }) comme dans les boucles for ou les blocs if.

let :

- A une portée de bloc. Cela signifie qu'elle est accessible uniquement à l'intérieur du bloc où elle a été définie (entre { ... }).
- Si utilisée dans une boucle ou un if, elle n'est pas accessible en dehors de ce bloc.

```
function testScope()
{
  if (true) {
    var x = 10;
    let y = 20;
  }
  console.log(x);
  console.log(y);
}
testScope();
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

20

20

Javascript : éléments de base

Déclaration multiple

var :

Vous pouvez déclarer plusieurs fois une variable avec le même nom dans la même portée, ce qui peut conduire à des comportements inattendus.

let :

Vous ne pouvez pas déclarer une variable avec le même nom dans la même portée.

```
var x = 10;
var x = 20;
console.log(x);

let x = 10;
let x = 20;
console.log(x);
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

21

21

Javascript : éléments de base

Déclaration des variables

En JavaScript, **var** et **let** sont deux mots-clés utilisés pour déclarer des variables

- **let** permet de déclarer des variables dont la portée est limitée à celle du bloc dans lequel elles sont déclarées.
- Le mot-clé **var**, quant à lui, permet de définir une variable globale ou locale à une fonction (sans distinction des blocs utilisés dans la fonction).

```
function exempleVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // x est visible ici,
  même si déclaré dans un bloc
}

exempleVar(); // Affiche 10
```

```
function exempleLet() {
  if (true) {
    let y = 20;
  }
  console.log(y); // Erreur : y n'est pas
  défini ici
}

exempleLet(); // Cette ligne génère une
  erreur
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

22

22

Javascript : éléments de base

Déclaration des variables

JavaScript langage avec typage dynamique

- type d'une variable défini qu'au moment de l'exécution
- possibilité de changer le type d'une variable à l'exécution

pas obligation d'initialiser une variable à sa déclaration

- une variable non initialisée a la valeur **undefined**

possibilité d'initialiser une variable à sa déclaration à l'aide d'expressions littérales

```
//Déclaration de a, de b et de c.
let a;
console.log(a);
let b,c;
//Affectation de a avec un entier.
a = 1;
console.log(a);
// Affectation de a avec une chaîne
a = "une chaîne de caractères";
console.log(a);
//forme littérale pour un entier
let quantite = 150;
console.log(quantite);
//forme littérale pour un réel.
let TVA = 19.6;
console.log(TVA);
//forme littérale pour un tableau d'entiers
let tab1 = [ 1, 2, 3, 4 ];
console.log(tab1);
//forme littérale pour un objet
let etud = { "nom": "Bolloli",
  "prenom": "Marco"
};
console.log(etud);
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

23

23

Javascript : éléments de base

Constante

possibilité de déclarer des constantes en JavaScript

utiliser **const** au lieu de **let**

```
const tauxTVA= 19.6;
let tauxReduc= 0;
let nbProduitsAchete= 0;
```

```
if (nbProduitsAchete>= 5) {
  // appliquer un taux de réduction de 25 %
  tauxTVA= 0.25;
```

une tentative de modification de la constante va provoquer une erreur d'exécution avec un message explicite sur la console

Utiliser des majuscules pour les noms de constantes

```
Const TAUX_TVA = 0.2;
```



Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

24

24

Javascript : éléments de base

Variables : Conversions numériques

- Relatifs aux valeurs et conversions des nombres, on trouve 4 fonctions :

- isNaN détermine si un paramètre donné n'est pas un nombre
- Number effectue une conversion
- parseInt effectue une conversion en valeur entière
- parseFloat effectue une conversion en valeur réelle

Exemple

- alert(isNaN(10));
- alert(isNaN("10"));
- alert(isNaN("blue"));
- let num1 = Number("hello world");
- let num2 = Number("00001");
- let num3 = Number(true);
- let num3 = parseInt("");
- let num4 = parseInt(22.5);
- let num6 = parseFloat("22.5");

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

25

25

Javascript : éléments de base

Variables : String

- On utilise les quotes simples (apostrophes) ou doubles (guillemets) ou ` (backticks) pour définir des valeurs chaînes de caractères. L'attribut length permet de déterminer la longueur d'une chaîne.

Exemple

- let nom = "Wilde";
- let prenom = 'Oscar';
- alert(prenom.length); // 6
- let message = "toto a dit \"je suis malade\".";
- let s1 = "Voilà l'automne";
- let s2 = `Il a dit : "Ah bon !"`;
- let nbHeures = 3;
- let s3 = `durée en secondes : \${nbHeures\*60} s`; // "durée en secondes : 180 s"

``{ ... }`` permet d'inclure des variables et expressions dans une chaîne.

La valeur de l'expression est évaluée et son résultat devient partie de la chaîne.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

26

26

Javascript : éléments de base

Variables : String

Il existe de nombreuses fonctions sur les chaînes de caractères.

Exemple

- let s = "hello world";
- alert(s.length); // 11
- alert(s.charAt(1)); // "e"
- alert(s.slice(3)); // "lo world"
- alert(s.slice(-3)); // "rid"
- alert(s.substring(3,7)); // "lo w"
- alert(s.indexOf("o")); // 4
- alert(s.lastIndexOf("o")); // 7
- alert(s.toUpperCase()); // HELLO WORLD
- alert(s + " !"); // hello world !
- Il est possible de transtyper une valeur en chaîne avec String().
- Exemple
- let v1 = 10; alert(String(v1)); // "10"
- let v2 = true; alert(String(v2)); // "true"
- let v3 = null; alert(String(v3)); // "null"
- let v4; alert(String(v4)); // "undefined"

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

27

27

Javascript : éléments de base

Opérateurs

- JavaScript propose les opérateurs et instructions standards de C++/Java

- Arithmétiques (+, -, *, /, %, ++, --, **)
- Comparaisons (==, !=, <, >, <=, >=, ===, !==)
- affectation (=, +=, -=, *=, /=, %=)
- Logiques (&&, ||, !)

Exemple

```
let age = 10;
age++;
alert(age);
alert(age > 10 && age < 20);
alert(26 % 5);
age*=2;
alert(age);
```

```
let a = 5;
let b = "5";

// L'opérateur === vérifie si les valeurs sont
strictement égales
if (a === b) {
  console.log("a est strictement égal à b");
} else {
  console.log("a n'est pas strictement égal à b");
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

[operateur.html](#)



28

Javascript : éléments de base

Conversion automatique des types

JavaScript effectue des conversions automatiques de types

- `50 * 20` -> 1000
- `'50' * '20'` -> 1000 // les valeurs de type string sont converties en valeurs de type number
- `'50' + '20'` -> '5020' //l'opérateur +est surchargé quand les opérandes sont de type number c'est l'addition, quand ils sont de type string c'est la concaténation de chaînes
- `'50' + 20` -> '5020' //l'un des opérandes est de type string ->+ est la concaténation de chaînes, la valeur de type number est convertie en valeur type string
- `50 + '20'` -> '5020' //l'un des opérandes est de type string ->+ est la concaténation de chaînes, la valeur de type number est convertie en valeur type string
- `'50' - 20` -> 30 //l'opérateur - est la soustraction, la valeur de type string est convertie en valeur type number
- `50 + null` -> 50 //la valeur null est convertie en la valeur 0 de type number
- `'50' + null` -> '50null' //la valeur null est convertie en la valeur 'null' de type string
- `50 + undefined` -> NaN

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

29

29

JAVASCRIPT



STRUCTURES DE CONTRÔLE

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

30

30

Structures de contrôle

- On utilise **if** pour exécuter du code si une condition est vérifiée.
- On peut ajouter un « sinon », appelé **else**, pour exécuter du code différent si la condition n'est pas vérifiée.

```
let age = parseInt(prompt("Donner votre age"));

if (age >= 18) {
  console.log("Vous pouvez conduire !");
}
else {
  console.log("Désolé, vous pourrez seulement conduire dans "
    + (18 - age) + "ans.");
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

31

31

Structures de contrôle

- Pour des conditions multiples, possible d'utiliser **else if** :

```
let age = 20;
if (age >= 24) {
  console.log("Vous pouvez voter et devenir sénateur !");
}
else if (age >= 18) {
  console.log("Vous pouvez voter.");
}
else if (age >= 16) {
  console.log("Vous ne pouvez pas voter mais vous pouvez "
    + "travailler sous certaines conditions.");
}
else {
  console.log("Vous devez être à l'école ?");
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

32

32

Structures de contrôle

- Possible de supprimer les accolades lorsqu'on n'a qu'une seule instruction :

```
let age = 20;
if (age >= 24)
  console.log("Vous pouvez voter et devenir sénateur !");
else if (age >= 18) {
  console.log("Vous pouvez voter.");
  console.log("Félicitations !");
}
else if (age >= 16)
  console.log("Vous ne pouvez pas voter mais vous pouvez"
    + "travailler sous certaines conditions.");
else
  console.log("Vous pouvez... aller à l'école ?");
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025



33

Structures de contrôle

Alternative

L'opérateur ternaire est une façon concise d'écrire une condition if-else. Il utilise le symbole ? et a la structure suivante :

`variable = condition ? valeur_si_vrai : valeur_si_faux;`

Voici un exemple simple :

`let civilite = (genre == "F") ? "Madame" : "Monsieur";`

Elle est équivalente à :

```
if (genre == "F")
  civilite = "Madame";
else
  civilite = "Monsieur";
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025



34

Structures de contrôle

Opérateurs logiques sur des booléens :

- Opérateur ET : a && b**
`true` si `a` et `b` tous les deux vrais, `false` sinon. Si `a` est `false`, JavaScript n'essaie même pas d'évaluer `b`, car il sait que l'expression ne pourra être vraie de toute façon : renvoie directement `false`.
- Opérateur OU : a || b**
`true` si soit `a` soit `b` vrai, `false` sinon. Si `a` est `true`, JavaScript n'essaie même pas d'évaluer `b`, car il sait que l'expression ne pourra être fausse de toute façon : renvoie directement `true`.
- Opérateur NON : !a**
Renvoie `true` si `a` vaut `false` et inversement.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

35

35

Structures de contrôle

JavaScript propose des opérateurs qui vous permettent de comparer des valeurs.

Opérateur	Description
<code>==</code>	Pour tester une égalité entre deux valeurs.
<code>===</code>	Pour tester une égalité entre deux valeurs en prenant aussi en considération le type des valeurs (chaîne, nombre, date).
<code>!=</code> ou <code><></code>	Pour tester la différence entre deux valeurs.
<code>!==</code>	Pour tester la différence entre des valeurs ou des types différents.
<code><</code>	Pour tester si une valeur est strictement inférieure à une autre.
<code><=</code>	Pour tester si une valeur est inférieure ou égale à une autre.
<code>></code>	Pour tester si une valeur est strictement supérieure à une autre.
<code>>=</code>	Pour tester si une valeur est supérieure ou égale à une autre.
<code>+=</code>	Pour concaténer. L'expression <code>a+=6</code> est équivalent à <code>a+=6</code> .

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

36

36

Structures de contrôle

Exercice 1 (Utiliser un IF ternaire):

Si une personne est âgée de 18 ans et plus c'est une personne majeure, sinon c'est une personne mineure.

Exercice 2 :

Ecrivez un programme en JavaScript qui demande à l'utilisateur de saisir une note (grade) comprise entre 0 et 100. En fonction de la note saisie, le programme doit afficher une alerte avec la mention correspondante :

- A si la note est supérieure ou égale à 90.
- B si la note est comprise entre 80 et 89.
- C si la note est comprise entre 70 et 79.
- D si la note est comprise entre 60 et 69.
- F si la note est inférieure à 60.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

37

37

Structures de contrôle

Choix multiple

L'instruction switch :

```
switch(x) {  
  case 1 :  
    instructions 1;  
    break;  
  case 2 :  
    instructions 2;  
    break;  
  ...  
  case n :  
    instructions 3;  
    break;  
  default : instructionsDefault;  
};
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

38

38

Structures de contrôle

Choix multiple

```
<script>  
let nb1, nb2;  
let op;  
nb1 = parseInt(prompt("donner nombre 1"));  
nb2 = parseInt(prompt("donner nombre 2"));  
op = prompt("donner operateur (+, -, *, /)");  
  
switch(op) {  
  case "+":  
    document.write(nb1+nb2);  
    break;  
  case "-":  
    document.write(nb1-nb2);  
    break;  
  case "*":  
    document.write(nb1*nb2);  
    break;  
  case "/":  
    {  
      if (nb2 == 0) document.write("Operation indéterminée");  
      else document.write(nb1/nb2);  
    }  
    break;  
  default:  
    document.write("Operation inexistante");  
}  
</script>
```

Pr. Ghazouani Mohamed

39

39

JAVASCRIPT



LES BOUCLES

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

40

40

Les boucles

- Boucles utilisées pour répéter du code plusieurs fois
- Utile pour ne pas réécrire la même chose de nombreuses fois, et souvent nécessaire car le nombre de fois qu'on veut répéter le code peut dépendre d'un paramètre dont la valeur ne peut être prédite
- Boucle **while** (« tant que ») : répète le code tant que la condition entre parenthèses est vraie

```
let bananas = 10;
while (bananas >= 1) {
  console.log ("J'ai " + bananas + " bananes et j'en mange une...");
  bananas--;
}
console.log ("Je n'ai plus de bananes.");
```
- Au passage, `bananas--` décrémente (= soustrait 1 à) la variable `bananas`, équivalent à `bananas = bananas - 1` ou `bananas -= 1`. De la même manière, `bananas++` incrémente (ajoute 1 à) la variable `bananas`.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

41

41

Les boucles

La boucle do...while

- La boucle `do...while` en JavaScript exécute le bloc de code au moins une fois, puis continue de l'exécuter tant que la condition spécifiée est vraie.
- Ce type de boucle peut être utile pour des scénarios où tu veux t'assurer qu'une certaine condition est remplie avant de continuer avec le reste de ton programme.

```
do
{
  // Le code à exécuter
}
while (condition);
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

42

42

Les boucles

À votre avis, quel est le problème de ce script ?

```
let bananas = 10;
while (bananas >= 1) {
  console.log ("J'ai " + bananas + " bananes et j'en mange une...")
}
```

- Boucle infinie !** Il faut toujours au moins que quelque chose « change » dans votre boucle pour qu'elle ait une chance de se terminer...
- C'est nécessaire mais pas suffisant ! Par exemple, si vous incrémentez la variable `bananas` au lieu de la décrémente, vous « changerez » quelque chose, mais quand même une boucle infinie...
- Page qui ne répond pas, peut-être un message d'erreur au bout d'un moment...

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

43

43

Les boucles

- Boucle **for** (« pour ») : généralement, répète le code pour un nombre de fois donné, e.g.

```
for (let i = 0; i < 100; i++)
  console.log (i);
```

`console.log ("J'ai compté de 0 à 99 !");`
- Une boucle **for** a 3 expressions séparées par des points-virgule :
 - for** (à exécuter au départ ;
 - test** : on reste dans la boucle ? (effectué aussi avant d'entrer) ;
 - à exécuter à la fin de chaque itération**)
- On peut mettre plusieurs instructions séparées par des virgules pour chaque expressions... Du coup on peut faire des choses plus compliquées :

```
for (let i = 0, j = 0; i < 100; i++, j += 2)
  document.write(i + " " + j + "<br />");
```

Au passage, remarquez que pour les **for** aussi on peut supprimer les accolades si on n'a qu'une seule instruction !
Marche pour les `if`, `else if`, `else`, `while`, `for`... à peu près pour n'importe quel bloc sauf les fonctions !

```
* break sort d'une boucle prématurément :
for (let i = debut; i <= fin; i++) {
  console.log("Je teste " + i); // Arrêt au premier multiple de 7
  if (i % 7 == 0) {
    console.log("Trouvé ! " + i);
    break; // Sort de la boucle
  }
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

44

Les boucles

Exercice "Fizz Buzz"

L'exercice du "Fizz Buzz" est un problème couramment utilisé dans les entretiens techniques pour évaluer les compétences de programmation de base des candidats.

- Commencez à partir de 1 et comptez jusqu'à 50.
- Pour chaque nombre dans la séquence, effectuez les vérifications suivantes :
 - Si le nombre est divisible par 3, affichez "Fizz" à la place du nombre.
 - Si le nombre est divisible par 5, affichez "Buzz" à la place du nombre.
 - Si le nombre est à la fois divisible par 3 et par 5, affichez "Fizz Buzz" à la place du nombre.
 - Sinon, affichez simplement le nombre lui-même.

Voici un exemple des premiers éléments de la séquence "Fizz Buzz" jusqu'à 15 :

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
9
Buzz
11
Fizz
13
14
Fizz Buzz
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

45

Les boucles

Exercice "Fizz Buzz" - Solution

```
for (let i = 1; i <= 50; i++)
{
    if (i % 3 == 0 && i % 5 == 0)
        console.log ("FizzBuzz");
    else if (i % 3 == 0)
        console.log ("Fizz");
    else if (i % 5 == 0)
        console.log ("Buzz");
    else
        console.log (i);
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

46



Les boucles

Exercice : Trouver la somme des nombres

Écrivez un programme JavaScript qui utilise une boucle `while` pour calculer la somme des nombres de 1 à un nombre donné.

Étapes à suivre :

- Demandez à l'utilisateur d'entrer un nombre positif (par exemple : 5).
- Utilisez une boucle `while` pour calculer la somme des nombres de 1 jusqu'à ce nombre inclus.
- Affichez le résultat final dans la console.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025



47

JAVASCRIPT



TABLEAUX

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

48

48

Les tableaux

- Les tableaux sont des listes ordonnées de valeurs. Par exemple :
`let tableau = [element0, element1, ...];`
- Vous pouvez mettre n'importe quel type de valeur dans un tableau, et des cases différentes d'un tableau peuvent avoir des valeurs différentes :
`let couleursArcEnCiel = ["Rouge", "Orange", "Jaune", "Vert", "Bleu", "Indigo", "Violet"];
let numerosDuLoto = [33, 2, 34, 18, 17, 45];
let nomAnneeNaissance = ["Antoine Martin", 1984];`
- Possible de connaître le nombre d'éléments d'un tableau avec la propriété `length` :
`console.log(couleursArcEnCiel.length);` // affiche 7

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

49

49

Les tableaux

- On peut accéder à une case d'un tableau en mettant son indice entre crochets. Par exemple, pour modifier un élément :
`nomAnneeNaissance[1] = 1985;`
- Pour lire un élément :
`console.log(nomAnneeNaissance[1]);`
- Les tableaux n'ont pas une taille fixe. Vous pouvez utiliser `push()` pour ajouter un élément à la fin d'un tableau :
`let langages = ["Java", "Javascript", "C++"];
langages.push("Fortran");`
/ Maintenant, langages contient ["Java", "Javascript", "C++", "Fortran"]. */*
- Pour récupérer le dernier élément d'un tableau et l'enlever : fonction `pop()`
`tab = langages.pop();`
/ tab vaut "Fortran" et langages contient ["Java", "Javascript", "C++"]. */*
- Tableaux en deux dimensions possible :
`let matrix = [[1, 2, 3],[4, 5, 6],[7, 8, 9]];`

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

50

50

Les tableaux

- `unshift(element1, element2, ...)`
Ajoute un ou plusieurs éléments au début du tableau.
`let fruits = ["banane", "orange"];
fruits.unshift("pomme");
console.log(fruits);
// ["pomme", "banane", "orange"]`
- `shift()`
Supprime et retourne le premier élément du tableau.
`let fruits = ["pomme", "banane", "orange"];
let premier = fruits.shift();
console.log(premier); // "pomme"
console.log(fruits);
// ["banane", "orange"]`
- `indexOf(element)`
Renvoie l'index de la première occurrence d'un élément, ou -1 s'il n'est pas trouvé
`let fruits = ["pomme", "banane", "orange"];
console.log(fruits.indexOf("banane"));
// Affiche : 1`
- `includes(element)`
Vérifie si un tableau contient un élément.
`let fruits = ["pomme", "banane", "orange"];
console.log(fruits.includes("banane"));
// true`

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

51

51

Les tableaux

Exercice 1 :

Écrivez un programme en JavaScript qui crée un tableau contenant tous les nombres entiers de 1 à 10. Utilisez une boucle pour ajouter chaque nombre au tableau.

Instructions :

- Déclarez un tableau vide nommé `nombre`.
- Déclarez deux variables, `start` et `end`, initialisées respectivement à 1 et 10.
- Utilisez une boucle `for` pour parcourir tous les nombres de `start` à `end` (inclus).
- À chaque itération de la boucle, ajoutez le nombre courant au tableau `'nombre'` en utilisant la méthode `push`.

Exercice 2 :

Écrivez un programme en JavaScript qui crée un tableau contenant tous les nombres pairs de 1 à 100. Utilisez une boucle pour ajouter chaque nombre pair au tableau.

Instructions :

- Déclarez un tableau vide nommé `nombre`.
- Utilisez une boucle `for` pour parcourir tous les nombres de 1 à 100 (inclus).
- À chaque itération de la boucle, vérifiez si le nombre courant est pair (utilisez l'opérateur modulo `%`).
- Si le nombre est pair, ajoutez-le au tableau `nombre` en utilisant la méthode `push`.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

52

52

Les tableaux

Exercice 3: Trouver la valeur maximale dans un tableau

Écrivez un programme JavaScript qui trouve la valeur maximale dans un tableau donné et l'affiche.

```
const tableau = [12, 45, 6, 78, 32, 56, 89, 23];
let max = tableau[0]; // Initialisez la variable max avec la première valeur
// du tableau

// Parcourez le tableau pour trouver la valeur maximale
for (let i = 1; i < tableau.length; i++) {
  if (tableau[i] > max) {
    max = tableau[i]; // Mettez à jour max si une valeur plus grande est
    trouvée
  }
}

console.log("La valeur maximale dans le tableau est : " + max);
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

53

53

Les tableaux

Concaténation de Deux Tableaux en JavaScript

Avant ES6

1. Utilisation d'une boucle pour ajouter les éléments:

```
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];
for(let i = 0; i < arr2.length; i++) {
  arr1.push(arr2[i]);
}
console.log(arr1); // Affiche [1, 2, 3, 4, 5, 6]
```

2. Utilisation de la méthode concat():

```
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];
let combined = arr1.concat(arr2);
console.log(combined); // Affiche [1, 2, 3, 4, 5, 6]
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

54

54

Les tableaux

Après ES6

1. Utilisation de l'opérateur de décomposition (...):

```
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];
let combined = [...arr1, ...arr2];
console.log(combined); // Affiche [1, 2, 3, 4, 5, 6]
let combined = [...arr1, 'Bonjour', ...arr2, 72];
console.log(combined); // Affiche [1, 2, 3, 'Bonjour', 4, 5, 6, 72]
//Utilisez l'opérateur de décomposition ... pour créer un nouveau tableau sans modifier les tableaux
originaux.
```

2. Utilisation de l'opérateur de décomposition avec push:

```
arr1.push(...arr2);
console.log(arr1); // Affiche [1, 2, 3, 4, 5, 6]
//Vous pouvez utiliser push(...arr2) pour ajouter des éléments, mais cela modifie le tableau original.
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

55

55

JAVASCRIPT



FONCTIONS

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

56

56

Les fonctions

- Une fonction correspond à un groupe de lignes de code, dont l'objectif est d'effectuer une tâche spécifique.
- Une fonction est nommée et réutilisable.
- Son nom commence par une lettre, ne contient ni espace ni caractères spéciaux et n'est pas un mot clé de JavaScript.
- Les fonctions se déclarent dans la partie `<script>` `</script>`.

La syntaxe est la suivante :

```
function nom_de_la_fonction()
{
    corps de la fonction
}
Par exemple :
function info()
{
    alert("Le message est bien envoyé");
}
```

On peut aussi intégrer des variables dans la définition de la fonction.

```
Function nom_de_la_fonction(var1,var2)
{
    corps de la fonction
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

57

57

Les fonctions

- Passage par valeur** : Lorsque vous passez une variable primitive (comme un nombre, une chaîne de caractères, un booléen, etc.) à une fonction, une copie de la valeur est créée. Toute modification apportée à cette copie n'affectera pas la valeur originale en dehors de la fonction.
- Passage par référence** : Lorsque vous passez un objet (comme un tableau, une fonction, ou un objet) à une fonction, une référence à l'objet est passée. Toute modification apportée à l'objet à l'intérieur de la fonction affectera l'objet original en dehors de la fonction.

```
let a = 10;
function changeValue(x) {
    x = 20;
}
changeValue(a);
console.log(a);
// Affiche 10
```

```
let tableau = [1, 2, 3];
function changeArray(a) {
    a[0] = 99;
}
changeArray(tableau);
console.log(tableau);
// Affiche [99, 2, 3]
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

58

58

Les fonctions

Fonction avec return

Une fonction avec return renvoie une valeur lorsqu'elle est appelée. Cette valeur peut être utilisée par le code qui appelle la fonction. Voici un exemple :

```
let num = 10;
function changeValue(x) {
    x = 20;
    return x;
}
let newNum = changeValue(num);
console.log(newNum); // Affiche 20
console.log(num); // Affiche 10
```

Dans cet exemple, la fonction `changeValue` change la valeur de `x`, et renvoie sa valeur. La valeur retournée est ensuite stockée dans la variable `newNum` et peut être utilisée plus tard.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

59

59

Les fonctions

Procédure vs Fonction

Méthode 1 :

```
nom_var ← nom_fonction (arg1 , arg2 , ... )
```

Méthode 2 :

```
nom_var1 ← nom_var2 / nom_fonction (arg1 , arg2 , ... ) ;
```

Méthode 3 :

```
Ecrire ( nom_fonction (arg1 , arg2 , ... ) )
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

60

60

Les fonctions

Procédure vs Fonction

Méthode :

```
nom_procedure (arg1 , arg2 , ... )
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

61

61

Les fonctions

Exercice 1 :

Écrivez une fonction `sumBetween(a, b)` qui prend deux nombres entiers `a` et `b` en arguments et retourne la somme de tous les nombres entiers compris entre `a` et `b` (inclusivement).

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

62

62

Les fonctions

Exercice 1 :

Écrivez une fonction `sumBetween(a, b)` qui prend deux nombres entiers `a` et `b` en arguments et retourne la somme de tous les nombres entiers compris entre `a` et `b` (inclusivement).

```
Solution :
function sumBetween(a, b) {
  let total = 0;
  for (let i = a; i <= b; i++) {
    total += i;
  }
  return total;
}
let resultat = sumBetween(2, 6);
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

63

63

Les fonctions

Exercice 2:

Écrivez une fonction `countLetters(letter, word)` qui prend deux arguments : une lettre `letter` et un mot `word`. La fonction doit retourner le nombre de fois que la lettre `letter` apparaît dans le mot `word`.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

64

64

Les fonctions

• Exercice 2:

Écrivez une fonction `countLetters(letter, word)` qui prend deux arguments : une lettre `letter` et un mot `word`. La fonction doit retourner le nombre de fois que la lettre `letter` apparaît dans le mot `word`.

Solution :

```
function countLetters(letter, word) {
  let counter = 0;
  for (let each_letter of word) {
    if (each_letter.toLowerCase() === letter) {
      counter++;
    }
  }
  return counter;
}

let resultat = countLetters("o", "Bonjour tout le monde !");
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

65

65

Les fonctions

Les Arrow Functions en JavaScript

- Les arrow functions sont une manière rapide et simplifiée d'écrire des fonctions en JavaScript.
- Elles sont apparues avec ES6 (ECMAScript 2015).
- Elles permettent d'écrire moins de code, tout en gardant la même fonctionnalité.

Fonction classique

Voici comment écrire une fonction classique qui calcule le carré d'un nombre :

```
function carre(nombre) {
  return nombre * nombre;
}
console.log(carre(4)); // 16
```

Fonction anonyme (sans nom) qui est assignée à la variable `carre`. `carre` devient un identificateur qui fait référence à cette fonction (avant l'introduction des arrow functions). Souvent, la fonction anonyme n'est utilisée qu'une seule fois, il n'est donc pas nécessaire de lui donner un nom.

```
const carre = function(nombre) {
  return nombre * nombre;
}
console.log(carre(4));
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

66

66

Les fonctions

Les Arrow Functions en JavaScript

Fonction fléchée équivalente

`const` indique que la variable `carre` est destinée à pointer vers cette fonction spécifique et ne sera pas réassignée à une autre fonction (ce qui est une bonne pratique pour éviter des erreurs).

Version complète avec parenthèses et accolades :

```
const carre = (nombre) => {
  return nombre * nombre;
}
console.log(carre(4)); // 16
```

Version simplifiée pour une seule ligne (sans accolades et return)

Si la fonction ne contient qu'une seule expression `return`:

```
const carre = nombre => nombre * nombre;
console.log(carre(4)); // 16
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

67

67

Les fonctions

Les Arrow Functions en JavaScript

Cette fonction est remontée en haut du fichier lors de l'exécution. (c'est ce qu'on appelle le "hoisting"). Cette fonction est hoistée, donc on peut l'appeler avant même qu'elle ne soit définie.

```
console.log(carre(4)); // Fonction accessible avant sa déclaration

function carre(nombre) {
  return nombre * nombre;
}
```

Ici, `carre` est une variable qui reçoit une fonction, donc elle n'existe pas avant son affectation. Les fonctions fléchées suivent le même principe, car elles sont aussi des expressions. Ici, la fonction n'est pas hoistée, donc elle doit être définie avant d'être utilisée.

```
console.log(carre(4)); // ❌ Erreur

const carre = (nombre) => {
  return nombre * nombre;
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

68

68

Les fonctions

Les Arrow Functions en JavaScript

Fonction sans paramètre

Si une fonction ne prend aucun paramètre, utilisez des parenthèses vides () :

```
const saluer = () => "Salut tout le monde !";  
console.log(saluer()); // Salut tout le monde !
```

Fonction avec plusieurs paramètres

Avec plusieurs paramètres, vous devez garder les parenthèses :

```
const ajouter = (a, b) => a + b;  
console.log(ajouter(3, 7)); // 10
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

69

69

Les fonctions

Les Arrow Functions en JavaScript

Exercices

1- Créez une fonction fléchée `estPositif` qui vérifie si un nombre est positif.

```
console.log(estPositif(10)); // true  
console.log(estPositif(-5)); // false
```

2- Écrivez une arrow function qui prend un nombre et retourne `true` si le nombre est pair, sinon `false`.

```
console.log(estPair(4)); // true  
console.log(estPair(7)); // false
```

3- Créez une arrow function qui prend une chaîne de caractères et retourne sa longueur.

```
console.log(longueur("JavaScript")); // 10  
console.log(longueur("Arrow Function")); // 14
```

4- Créez une arrow function qui prend une liste de mots et retourne le mot le plus long.

```
console.log(motLePlusLong(["chat", "éléphant", "souris"])); // "éléphant"
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

70

70